

DataAcrossBorders

Project Analysis & Build Plan — Federated Medical Imaging Search

Repository: [iknalos/DataAcrossBorders](#) | Prepared: July 25, 2026 | Event: The Open Accelerator Healthcare Hackathon — Track 1

1. Executive Summary

The repository currently contains only the **hackathon boilerplate**: a deliberately minimal FastAPI application that simulates one siloed hospital database node. Run three times on different ports, it mimics three disconnected Boston hospitals (BCH, MGH, BWH), each holding 900 synthetic radiology study records with **no authentication, no search, and fully exposed patient PII**.

Nothing of "your solution" exists yet. The entire challenge deliverable — a federated search layer with privacy protection and access control that sits on top of the three nodes — still needs to be built. This document assesses what is provided, defines exactly what is missing, proposes an architecture, and breaks the work into a prioritized, hackathon-sized task list.

The one-sentence goal: Build an aggregator service + web UI that lets an authorized clinician search all three hospitals at once (e.g., "hydrocephalus in patients under 10") while redacting PII according to the requester's role, and log every access.

2. What the Repo Contains Today

File	Purpose	State
<code>main.py</code>	FastAPI node: <code>/health</code> , <code>/api/studies</code> , <code>/api/studies/{id}</code> . Loads one JSON file based on <code>HOSPITAL_NODE</code> env var.	Complete (65 lines) — do not modify per hackathon rules
<code>models.py</code>	Pydantic <code>StudyRecord</code> schema — 12 string fields incl. PII (name, DOB) and a full radiology report in <code>Diagnosis</code> .	Complete
<code>data/*.json</code>	3 × 900 synthetic studies (300 brain / 300 heart / 300 fetal each). BCH = pediatric (0–21), MGH = adults (22–85), BWH = adults (18–75). Conditions overlap across hospitals.	Complete
<code>README.md</code>	Challenge description, API reference, data schema.	Complete
<code>PRE_HACK_SETUP.md</code>	Environment setup guide (Python, Git, venv).	Complete
<code>requirements.txt</code>	fastapi, uvicorn, pydantic.	Complete

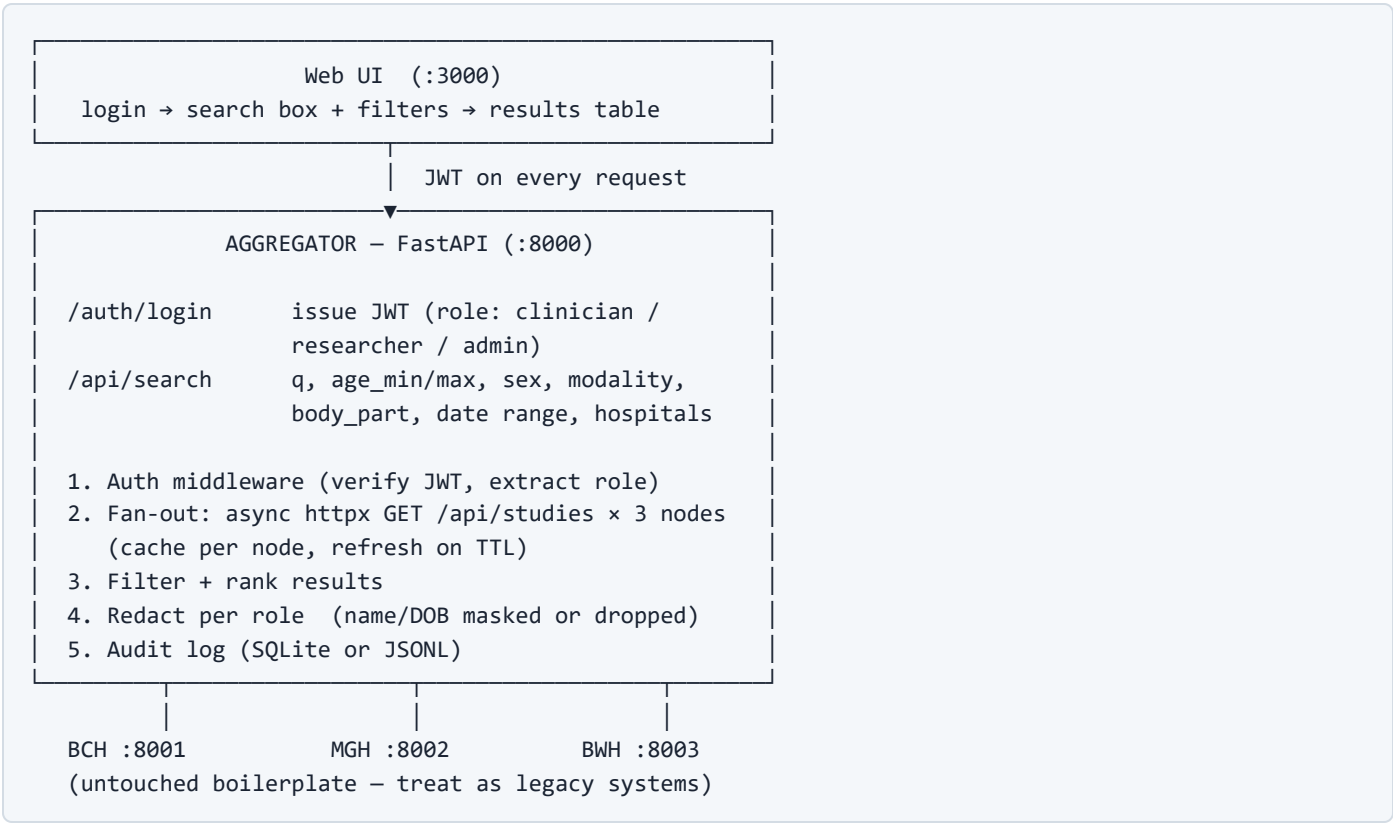
Note: The README's architecture section references a `scripts/generate_data.py` folder that is **not actually present** in the repo — harmless (it was the organizers' data generator), but be aware the tree diagram doesn't match the actual contents.

3. Gap Analysis — What Needs to Be Done

The boilerplate's intentional deficiencies define your work items directly:

Deficiency (by design)	What you must build
No search endpoint — only "return all 900 records"	Search & filter engine: free-text over Diagnosis , plus structured filters (age, sex, modality, body part, date range)
Nodes are unaware of each other	Federated aggregator that fans out queries to all three nodes in parallel and merges results
Zero authentication	Auth layer (login + roles), e.g. JWT with clinician / researcher / admin roles
PII fully exposed (names, birthdates)	Redaction / de-identification layer applied per role before results leave the aggregator
No audit trail	Access log: who searched what, when, and which records were returned
No UI	Search front end (simple web page is enough) for the demo

4. Proposed Architecture



Key design decisions

- **Don't touch the nodes.** Treat them as immutable legacy systems — that is the spirit of the challenge and impresses judges more than hacking the boilerplate.
- **Fetch-and-cache, filter in the aggregator.** The nodes only offer "give me everything" (2,700 records total, tiny). Pull all studies with `httpx.AsyncClient` + `asyncio.gather`, cache in memory with a short TTL, and run search locally. Graceful degradation: if one node is down, return results from the other two with a per-node status flag.
- **Role-based redaction matrix** (see Section 5). Redaction happens in one place — a single `redact(record, role)` function — so it is easy to demo and audit.
- **Age handling:** `PatientAge` comes as `007Y` / `011M` / `005D`. Normalize to a float in years at ingest so age-range filters work across pediatric and adult data.

5. Privacy / Redaction Design

Field	Admin	Clinician	Researcher
PatientName	full	full	REDACTED
PatientID	full	full	salted hash (consistent pseudonym)

PatientBirthDate	full	year only	REDACTED (keep PatientAge)
PatientAge / Sex	full	full	full (needed for research)
StudyInstanceUID	full	full	hashed
Diagnosis	full	full	full (synthetic text contains no names)

Stretch: run a lightweight PII scrubber (regex for `LastName^FirstName` patterns, dates) over `Diagnosis` text as defense-in-depth, and mention HIPAA Safe Harbor's 18 identifiers in the pitch — judges at healthcare hackathons reward teams that name the actual compliance standard.

6. Prioritized Task List

#	Priority	Task	Est.
1	P0	Scaffold aggregator (<code>aggregator/main.py</code>), async fan-out to 3 nodes with <code>httpx</code> , merge into one list with <code>hospital</code> source tag	1–1.5 h
2	P0	<code>/api/search</code> : free-text match over Diagnosis + filters (age range via normalized age, sex, modality, body_part, hospital, date range), pagination	1.5 h
3	P0	JWT auth: <code>/auth/login</code> with 3 hardcoded demo users (one per role), auth dependency on all search routes	1 h
4	P0	Role-based redaction function per matrix in Section 5	45 min
5	P1	Web UI: login page + search page (results table showing hospital badge, redaction visibly different per role). Plain HTML/JS or React — plain is faster	2 h
6	P1	Audit log (SQLite/JSONL): user, role, query, timestamp, result count + <code>/api/audit</code> viewer for admin role	45 min
7	P1	Resilience: per-node timeout, "2 of 3 hospitals responded" status in response envelope; demo by killing one node live	30 min
8	P2	Relevance ranking (term frequency in Diagnosis) + result highlighting	45 min
9	P2	Aggregate analytics endpoint: counts by condition/hospital/age-band with k-anonymity floor (suppress buckets < 5) — great judge bait	1 h
10	P2	Diagnosis-text PII scrubber (regex defense-in-depth)	30 min
11	P3	Semantic search over Diagnosis (embeddings, e.g. sentence-transformers or an API) — only if time remains	2 h+
12	P3	Repo hygiene: update README with solution architecture, add <code>run_all.ps1</code> / <code>run_all.sh</code> to launch 3 nodes + aggregator + UI in one command	30 min

MVP = tasks 1–4 (about half a day): a secured, role-aware federated search API demonstrable with curl/Swagger. Tasks 5–7 make it demo-ready. Tasks 8–10 differentiate you from other teams.

7. Suggested Repo Layout for the Solution

```

DataAcrossBorders/
├─ main.py, models.py, data/      # boilerplate nodes (unchanged)
├─ aggregator/
│   └─ main.py                    # FastAPI app :8000
│   └─ auth.py                    # JWT issue/verify, demo users, roles

```

```

├── federation.py          # async fan-out + cache + node health
├── search.py             # filters, text match, age normalization
├── redaction.py          # redact(record, role)
├── audit.py              # audit log write/read
├── ui/                   # static HTML/JS or small React app
├── run_all.ps1           # start nodes 8001-8003 + aggregator + UI
└── requirements.txt      # + httpx, python-jose, passlib

```

8. Demo Script (for judging)

1. Show a raw node: `curl :8001/api/studies` → point out exposed names/birthdates and no auth. "This is healthcare data today."
2. Log in as **researcher**, search "hydrocephalus" → results from all 3 hospitals, PII redacted, pseudonymized IDs.
3. Log in as **clinician**, same search → names visible, birthdate reduced to year. Same data, different lens.
4. Kill the MGH node mid-demo → search still returns BCH+BWH with a degraded-status banner. Resilience.
5. As **admin**, open the audit log → every query above is recorded.
6. Close with the analytics endpoint (condition counts across hospitals, small buckets suppressed) → "research insight without ever moving or exposing patient data."

9. Setup on Your Laptop (Windows 11 + conda)

You already have conda, so skip the venv instructions in `PRE_HACK_SETUP.md` and use this instead. Run everything from PowerShell.

One-time setup

```

# 1. Create and activate a dedicated env (Python 3.11)
conda create -n borders python=3.11 -y
conda activate borders

# 2. Install dependencies
cd C:\Users\test\Projects\DataAcrossBorders
pip install -r requirements.txt

# 3. Extra packages you'll need for the aggregator (Section 6)
pip install httpx "python-jose[cryptography]" "passlib[bcrypt]"

```

Running the three hospital nodes (three PowerShell windows)

```

# Window 1 – Boston Children's
conda activate borders; cd C:\Users\test\Projects\DataAcrossBorders
$env:HOSPITAL_NODE="BCH"; uvicorn main:app --port 8001 --reload

# Window 2 – Mass General
conda activate borders; cd C:\Users\test\Projects\DataAcrossBorders
$env:HOSPITAL_NODE="MGH"; uvicorn main:app --port 8002 --reload

```

```
# Window 3 – Brigham and Women's
conda activate borders; cd C:\Users\test\Projects\DataAcrossBorders
$env:HOSPITAL_NODE="BWH"; uvicorn main:app --port 8003 --reload
```

Verify

```
curl.exe http://localhost:8001/health # {"status":"healthy","node":"BCH"}
curl.exe http://localhost:8002/health # MGH
curl.exe http://localhost:8003/health # BWH
```

Note: in PowerShell use `curl.exe` (plain `curl` is an alias for `Invoke-WebRequest`). Interactive docs live at `http://localhost:8001/docs` (and 8002/8003). Once the aggregator exists it runs the same way on port 8000: `uvicorn aggregator.main:app --port 8000 --reload`. Task 12's `run_all.ps1` launches all of this in one command — write it early, it pays off every restart.

10. Risks & Gotchas

- `PatientAge` mixed units (`Y / M / D`) will silently break naive age filters — normalize first (BCH has neonates like `005D`).
- `PatientName` includes non-name values like `Smith^BabyBoy` — fine, but don't build name parsing that assumes real first names.
- Windows: env var syntax for the nodes differs (`$env:HOSPITAL_NODE="BCH"` in PowerShell); put it in `run_all.ps1` so it never bites during the demo.
- The aggregator must not double-count: StudyIDs are unique per node but the same *prefix scheme* (`BR-`, etc.) appears across hospitals — always key on `(hospital, StudyID)`.
- Nodes allow CORS `*`, but the UI should still talk only to the aggregator — going straight to nodes from the browser would bypass your entire privacy layer (worth stating out loud to judges).

DataAcrossBorders — analysis generated July 25, 2026. Source: repository inspection of `iknalos/DataAcrossBorders` (8 files, single commit). Est. total effort: MVP ~4.5 h, full demo build ~8–9 h.